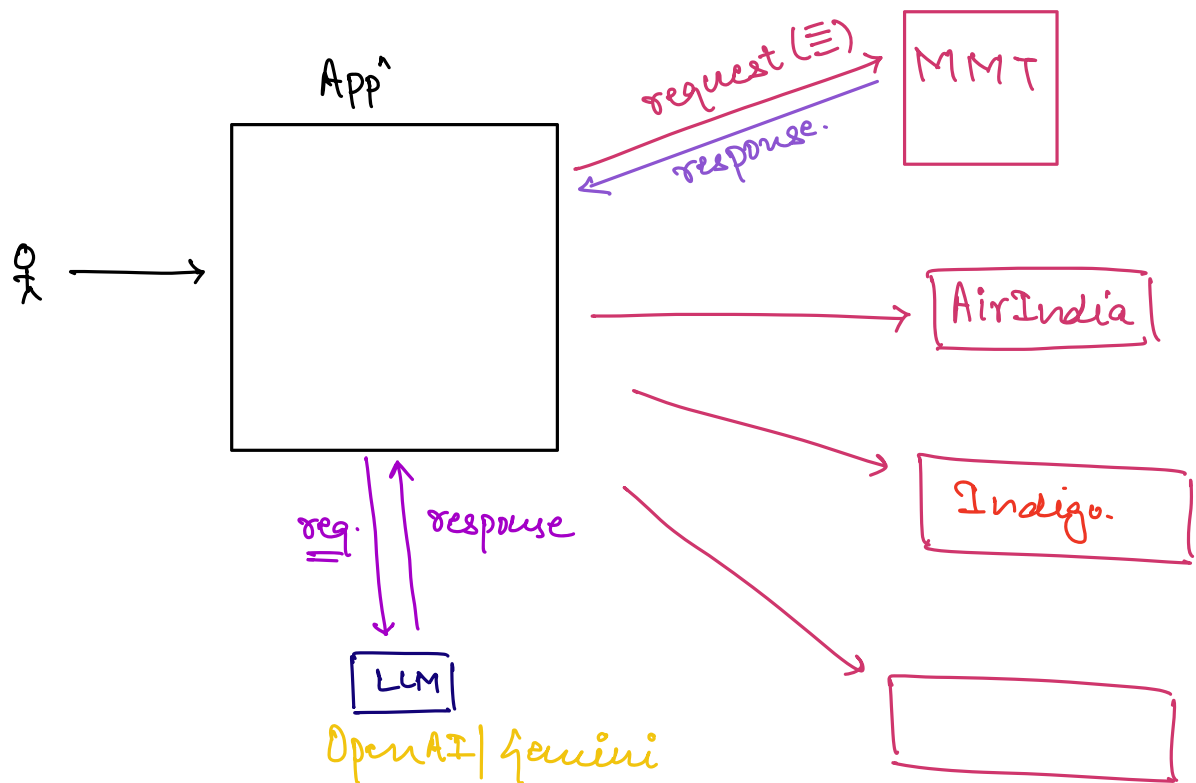


→ APLs

→ JSON.

⇒ AI Travel Agent.



API.

↳ Application Program ming Interface.



Client = OpenAI()

Client.responses.create (

model = "gpt-5.2",
instructions = "____",
input = "____"

>

APIs are the bridge between the Agent's brain & real-world software applications.

An API is a contract b/w two software components that defines what request will be sent & what response will be received.

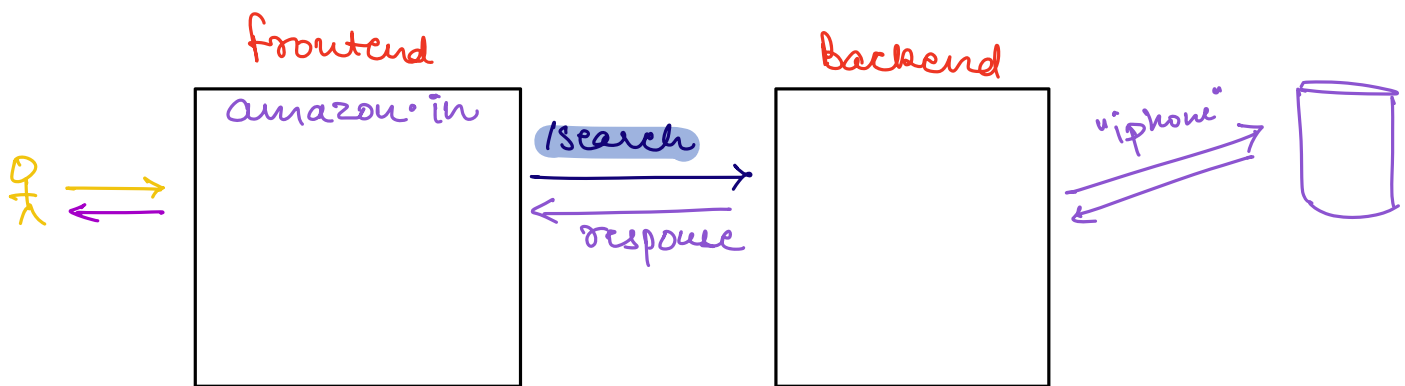
An API is a structured way for two systems to talk to each other.

How any modern Software Appⁿ is structured?

→ Frontend ^{Website} ^{Mobile Appⁿ}

→ Backend

→ Database.



https://www.amazon.in | s ? q = "iphone"

Domain.

API Endpoint.

Query Param.

API: Show my orders.

User clicks on "My Orders"



Frontend sends the API request



Backend receives the API request



Backend validates the User.



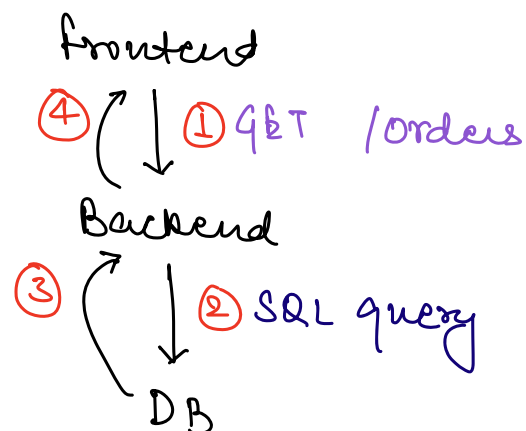
Backend sends a query to DB



DB Sends the response to Backend



Backend sends the response back to Frontend.



Response $\Rightarrow$ list of orders.

[

{
 "id": — ,
 "date": —
 " — " : —
 — : —
}

,

{
 "id": — ,
 "date": —
 " — " : —
 — : —
}

,

{
 "id": — ,
 "date": —
 " — " : —
 — : —
}

,

{
 "id": — ,
 "date": —
 " — " : —
 — : —
}

,

]

REST APIs.

↳ Representational State Transfer.

→ how to design & structure our APIs?

/orders

/payments

/users

/students

⇒ REST principle: APIs should be structured around the resources.

⇒ API $\begin{matrix} \swarrow \\ \text{C} \\ \swarrow \\ \text{R} \\ \swarrow \\ \text{U} \\ \swarrow \\ \text{D} \end{matrix}$

/createuser

/updateuser

/getuser

/deleteuser

/users $\begin{matrix} \swarrow & \text{GET} \\ \text{---} & \text{POST} \\ \swarrow & \text{PUT} \\ & \underline{\underline{\text{DELETE.}}} \end{matrix}$

GET /users → Read

↳ fetch resource.

POST /users → Create

↳ Add a new resource.

PATCH /users → Update.

↳ Partial Update.

PUT /users → Update.

↳ Replace.

DELETE /users → delete.

GET Course details.

→ GET /courses ⇒ get details of all the courses.

→ GET /courses/123 ⇒ get details of the course
with id = 123.

POST

POST /orders
↳ Request Body

{
 "order-id" : 123 ,
 "date" : _____ ,
 "status" : _____ ,

}

3

PATCH /PUT /users/123

↳ Request Body

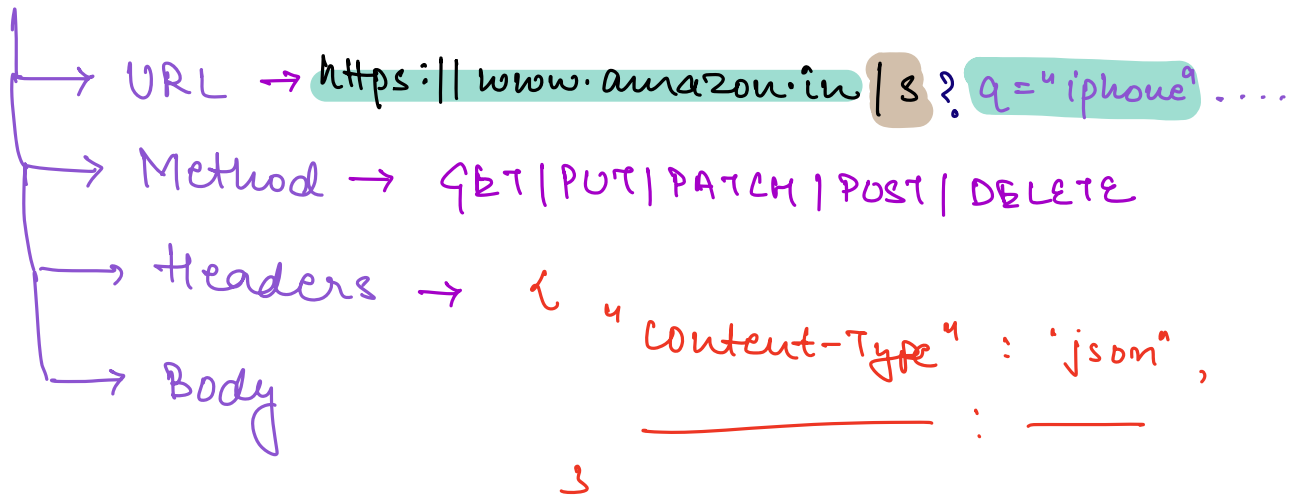
{
 "phone-no" : _____ ,
 "address" : _____ ,

}

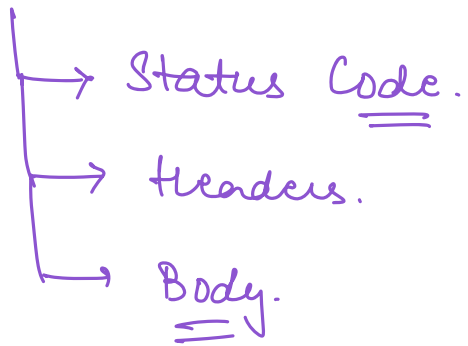
3

DELETE /users/171

Structure of API request.



API response.



HTTP Status Code.

200 $\Rightarrow$ OK : Request Successful.

201 $\Rightarrow$ New resource created.

400 $\Rightarrow$ Bad request.

401 $\Rightarrow$ Unauthorized.

500 $\Rightarrow$ Internal Server Error.

2xx → Success

4xx → Client's mistake.

5xx → Server's mistake.

JSON.

→ JavaScript Object Notation

→ lightweight text format used to send structured info b/w Systems.

```
{  
  "order-id" : 123 ,  
  "date" : _____ ,  
  "status" : _____ ,  
  _____  
  _____  
  _____  
}
```

→ Easy to read & parse.

```
"{  
  "order-id" : 123 ,  
  "date" : _____ ,  
  "status" : _____ ,  
  _____  
  _____  
  _____  
}"
```

→ Object.

import json

→ json.loads(): Decode JSON string

→ json.dumps(): encodes python object into json string.

```
{  
  "name": "Rahul",  
  "age": 25,  
  "price": 499.99,  
  "is_active": true,  
  "address": null,  
  "skills": ["Python", "APIs", "AI"],  
  "profile": {  
    "city": "Bangalore",  
    "country": "India"  
  }  
}
```

Parsing

→ Converting JSON string into Python object.

json.loads()

Stringifying JSON in Python

→ Convert Python dictionary into JSON.

json.dump()